AdCoherent Labs

SaaS Backend Architecture

Credit-metered, tier-rate-limited, security-first backend for the four-layer ad-compliance engine — an implementation specification for Claude Code.

Field	Value
Product	AdCoherent Labs — US fintech ad-compliance testing SaaS
Scope	Backend API, credit system, tier rate limiting, queue/worker, billing, security & privacy
Framework decision	FastAPI (Python 3.11) — single backend service + worker, one codebase (§2, §4)
Builds on	Existing 4-layer engine (ENGINE.md), self-hosted Supabase migrations 0001–0007 (supabase.md), Build Playbook Phases 4/6/8/9
Locked decisions	Flat credit cost by tier × asset type · monthly allowance + purchasable packs · Docker on this machine now, Linux VPS at launch · backend/ package in this repo
Status	v1.0 — approved for implementation · July 13, 2026

Read me first
This document is the single source of truth for the backend build. Engine internals (Layers 1–4) are DONE and must not be modified — the backend wraps `engine.saas` and `run_compliance_engine`. Existing Supabase migrations 0001–0007 are applied; new work starts at 0008. Every phase in §17 ends with a verification gate — do not start the next phase until the gate passes.

Contents

1 Purpose, scope & ground rules
2 Decisions locked (with rationale)
3 System topology & trust boundaries
4 Framework: why FastAPI
5 Repository layout
6 The request lifecycle, end to end
7 Data model additions (migrations 0008–0010)
8 The credit system
9 Rate limiting & abuse control
10 Job queue & worker design
11 API surface
12 Billing: Stripe integration
13 Security architecture (cyber safety)
14 Data privacy & compliance
15 Observability, reliability & ops
16 Testing & verification
17 Build order for Claude Code (phases & gates)
A Appendix A — Environment variables
B Appendix B — Seed tables (credits, limits, packs)
C Appendix C — Non-goals & hard guardrails

1 Purpose, scope & ground rules

AdCoherent Labs reviews US fintech marketing creatives (banner images, ad copy, HTML pages, carousel ads) and returns an auditable compliance verdict from a four-layer engine (CV → NLP → LLM+RAG → deterministic rule engine). The engine is complete and verified end-to-end (see ENGINE.md §10). This document specifies the **backend that turns the engine into a SaaS**: a secured API, a credit-based usage currency, tier-based rate limits, an async job pipeline, Stripe billing, and the privacy/audit posture a compliance product must itself satisfy.

In scope

- A FastAPI service (`backend/` in this repo) exposing the app API and the Pro+ programmatic API.
- A worker service executing the engine off a durable Postgres queue, with retries, dead-lettering, and per-tier concurrency.
- A credit ledger replacing the scans/month quota (migration 0008), wired to Stripe subscriptions and one-time credit packs.
- Tier-based rate limiting (Redis), upload security pipeline (MIME sniff + ClamAV), audit and privacy controls.
- Deployment as Docker Compose containers joined to the existing self-hosted Supabase network; identical compose moves to a Linux VPS at launch.

Out of scope

- Any change to engine internals: `CV-RULE_ENG-NLP/`, `rag-engine/`, and `engine/` layer logic are frozen. The backend consumes their public contracts only.
- Frontend implementation (the Next.js 16 app in `frontend/` consumes this API; endpoint contracts in §11 are what it codes against).
- Non-US jurisdictions, SSO/SAML build-out (Enterprise entitlement flag exists; implementation deferred), and the post-production OCR swap (one-dict change per ENGINE.md §3).

Ground rules for the implementing agent

- **Reuse, do not rewrite.** `engine/saas.py` already implements tenancy resolution, entitlement fetch, run persistence, findings, token metering, API keys, and hash-chained audit. Extend it; replace only its quota call (§8.5).
- **Forward-only migrations**, idempotent SQL (`if not exists` / `drop policy if exists`), applied via `supabase/apply.sh`, numbered from 0008.
- **Every gate in §17 must pass before the next phase begins.** The RLS isolation test and the credit race test are red-line CI checks: a failure blocks merge.
- **The DB decides, never the caller:** tier from `app.org_tier()`, org from memberships, credits from the ledger. A client-supplied org_id or tier is never authority.

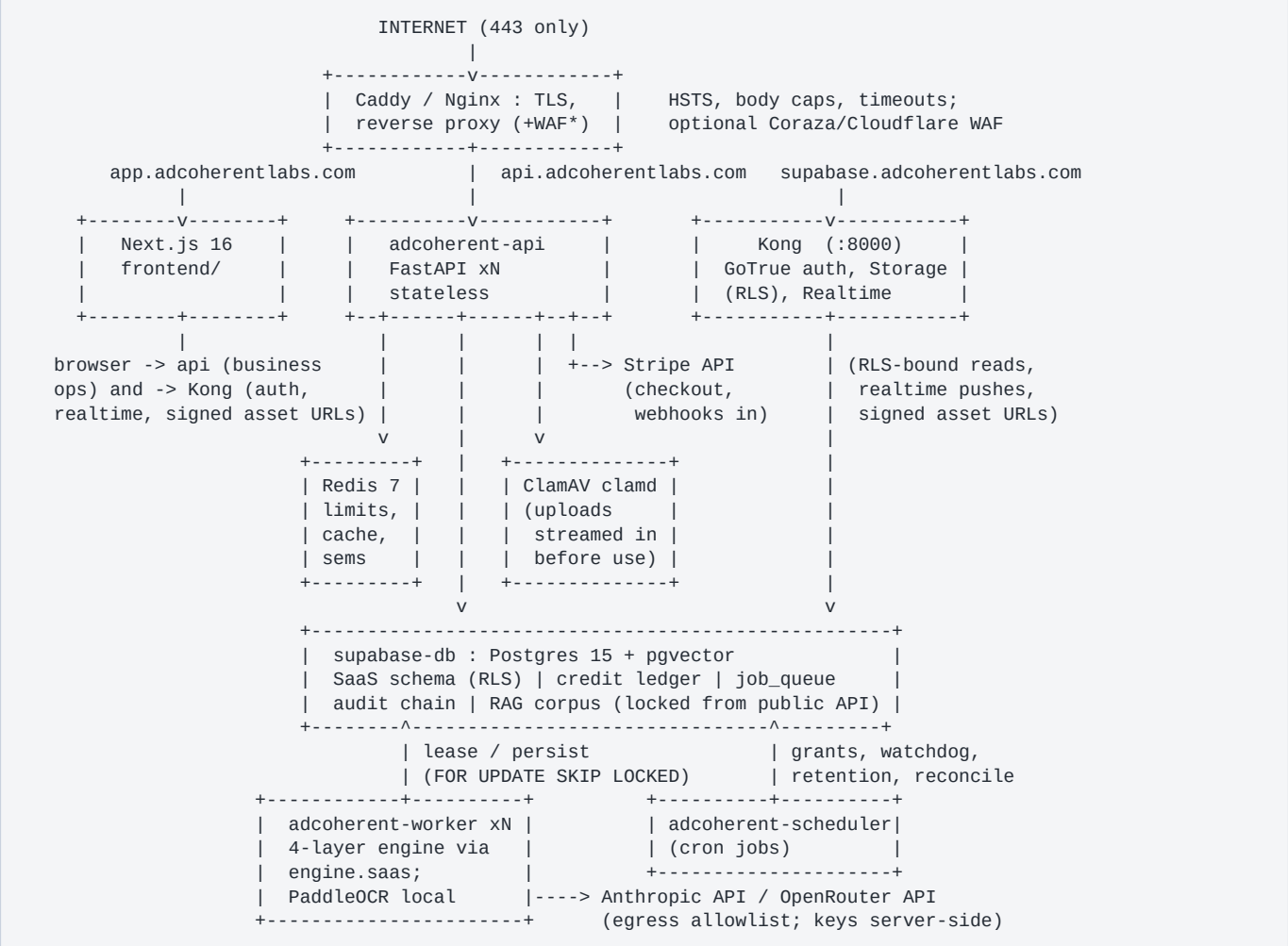
> **Launch-blocking prerequisite (do this before any backend code runs against the stack)**
> The moved Supabase stack still carries PUBLIC DEFAULT secrets (JWT_SECRET, POSTGRES_PASSWORD, SECRET_KEY_BASE, VAULT_ENC_KEY) and previously bound Kong/pooler on 0.0.0.0 (supabase.md §2–§3). Run `python supabase\harden.py` first: it rotates all secrets, mints new anon/service keys, loopback-binds ports, and verifies old keys are dead. All backend env files are written AFTER this rotation.

2 Decisions locked (with rationale)

#	Decision	Choice	Rationale (short)
D1	Backend framework	**FastAPI, Python 3.11**	Engine is Python; one language means auth/credit/audit logic exists exactly once; pydantic contracts shared with engine; auto-OpenAPI for the Pro+ API. Full case in §4.
D2	Service shape	One codebase, three processes: api / worker / scheduler	Same image, different entrypoints. API stays stateless; engine never runs in the request path.
D3	Queue	Postgres-backed (FOR UPDATE SKIP LOCKED)	Credit spend + run row + enqueue commit in ONE transaction — no orphan charges. No new broker to secure/back up. Redis is rate-limit/cache only. §10.
D4	Credit model	Flat cost = base(asset kind) × multiplier(tier model class)	User-selected. Predictable, published pricing; enforceable BEFORE expensive work; tokens still metered for margin analytics.
D5	Out-of-credits posture	Monthly allowance (resets, no rollover) + purchasable packs (never expire) + hard stop at zero	User-selected. Packs consumed after allowance; packs for paid tiers only.
D6	Deployment	Docker Compose on this Windows box (dev) → identical compose on hardened Linux VPS (launch)	User-selected. Containers join the existing supabase docker network; §15.4 has the VPS path.
D7	Code location	backend/ package in this repo	User-selected. Native import of engine + shared venv contracts; one CI.
D8	Rate-limit store	Redis 7 (container), atomic Lua token bucket	Sliding limits across N api replicas; in-process fallback if Redis is down (§9.4).
D9	Auth	Supabase JWT (HS256, local verify) for browser; sha256 API keys (existing engine/saas.py impl) for programmatic	No network hop per request; API-key plumbing already built and tested.
D10	Realtime to UI	Existing Supabase Realtime publication (analysis_runs, findings) under RLS	Already wired by migration 0005 + engine status updates; frontend subscribes, never polls.

Tier → model routing is already the engine's single source of truth and is unchanged: free = qwen/qwen-turbo, hobby = qwen/qwen-max (both via OpenRouter), pro = claude-sonnet-5, design = claude-opus-4-8, enterprise = claude-opus-4-8 (custom scale) — per `plan_entitlements` and `engine/tiers.py`. The playbook's Appendix A (Pro = Opus 4.7) was superseded by product decision; the DB seed is authoritative.

3 System topology & trust boundaries



Container	Image / basis	Role	Exposed
adcoherent-api	backend/Dockerfile (python:3.11-slim)	FastAPI app: auth, submissions, credits, billing, keys, webhooks	Via proxy only
adcoherent-worker	same image, worker entrypoint	Leases jobs, runs the engine, persists results, heartbeats	None
adcoherent-scheduler	same image, scheduler entrypoint	Credit cycle grants, stuck-run watchdog, retention deletion, ledger reconcile, audit-chain verify	None
redis	redis:7-alpine (password, no persistence needed)	Rate-limit counters, entitlement/JWT cache, org concurrency semaphores	Internal only
clamav	clamav/clamav:stable	clamd daemon; every upload streamed through it before processing	Internal only
(existing) supabase-*	self-hosted stack at C:\Users\HP\supabase-project	Postgres+pgvector, GoTrue, Storage, Realtime, Kong, pooler	Kong via proxy

Trust boundaries (where to be paranoid)

- **Browser ↔ API:** every request authenticated and org-resolved server-side; client-supplied org_id is never authority (it may only select among the caller's own memberships, re-verified in DB — the existing engine/saas.py contract).

- **User asset ↔ engine:** the most dangerous boundary. Files are hostile until MIME-sniffed, size-capped, malware-scanned; ad text is prompt-injection input — it enters Layer 3 as inert JSON data and Layer 4 deterministically re-verifies every answer (existing engine posture, ENGINE.md §7).
- **Backend ↔ model providers:** only extracted text and measurements leave (OCR is local PaddleOCR); model IDs pinned; keys server-side only; egress allowlist.
- **Tenant ↔ tenant:** RLS on every tenant table (migrations 0002/0006, tested) for user-token paths; the backend's privileged DB role bypasses RLS, so every statement scopes org_id explicitly — defence in depth, verified by the red-line CI test.
- **Public internet ↔ data plane:** only the reverse proxy listens publicly. Postgres (5432/6543/54322), Studio, Redis, clamd are never exposed; Kong/pooler stay loopback-bound after harden.py.

4 Framework: why FastAPI

The requirement is a security-critical, audit-heavy API in front of a Python ML engine, sold to US fintech marketing/compliance teams who will integrate it programmatically (Pro+ API is a paid entitlement). Evaluated against Next.js route handlers (playbook's hybrid default), NestJS/Express, Django REST, and Go:

- **The worker cannot be anything but Python** — it imports `run_compliance_engine`, PaddleOCR, spaCy, psycopg, pgvector natively. A Node/Go API would split the backend into two runtimes and force auth, org resolution, entitlements, credit charging, and audit logging to exist twice. For a solo builder on a security-critical product, duplicated security logic is the primary drift-and-vulnerability risk. One FastAPI codebase makes those checks exist exactly once, unit-tested once.
- **Contracts are already pydantic.** `EngineRunResult`, `AssetRepresentation`, and the review documents are pydantic models. FastAPI validates requests/responses with the same library — the engine contract IS the API contract, no translation layer to drift.
- **Auto-generated OpenAPI** gives the Pro/Design/Enterprise programmatic tier professional, always-current docs (fintech engineering teams evaluate APIs by their docs), plus typed TS client generation for the Next.js frontend — the shared-schema benefit the playbook wanted from Zod, without a second schema language.
- **Async-first request handling** (Starlette/uvicorn) fits the workload: the API is IO-bound (Postgres, Redis, Storage, Stripe) and returns 202 immediately for analysis; slow LLM work never blocks the request path.
- **Operationally boring**: one Docker image for api/worker/scheduler, uvicorn behind the existing proxy, horizontal scale by replica count.

Division of labour with Next.js: the existing `frontend/` app keeps Supabase for what it is good at — GoTrue sign-in/up flows, Realtime run-status subscriptions, and short-lived signed asset URLs, all under RLS with the anon key. Every business mutation (submit, approve, buy, invite, keys) goes through FastAPI, which holds the privileged role and enforces role + entitlement + credits + rate limits server-side. Stripe webhooks terminate at FastAPI, not Next.js.

Stack layer	Technology	Notes
Language / runtime	Python 3.11 (matches the engine venv)	Linux containers; PaddleOCR/paddle wheels install cleanly on python:3.11-slim + libgomp
Web framework	FastAPI + uvicorn (uvloop)	2 workers per api container; scale by replicas
Validation	pydantic v2 strict mode	reject unknown fields; shared with engine models
DB access	psycopg 3 (sync in worker, async pool in API)	API: pooler :6543 (transaction mode). Worker/scheduler: session port :5432. Migrations/DDL: socat :54322 direct
Cache / limits	redis-py, atomic Lua scripts	§9
Jobs	Postgres job_queue + lease functions	§10
Billing	stripe-python, webhook signature verify	§12
Uploads	python-magic (MIME sniff), Pillow (image re-encode), clamd stream scan	§13.3
Observability	structlog JSON, sentry-sdk, prometheus-client	§15
Tests	pytest + httpx AsyncClient + testcontainers-style compose	§16

5 Repository layout

New top-level `backend/` package beside the engine repos. Nothing inside existing folders changes except one call-site swap in `engine/saas.py` (§8.5) and new SQL under `supabase/migrations/`.

```
adcoherentlabs-project/
  engine/                      # FROZEN - orchestrator + saas data plane (extend saas.py only per sec.8.5)
  CV-RULE_ENG-NLP/             # FROZEN - Layers 1,2,4
  rag-engine/                  # FROZEN - Layer 3 + corpus
  frontend/                    # Next.js 16 app (consumes this API; not in backend scope)
  supabase/
    migrations/
      0001..0007_*.sql         # applied
      0008_credits.sql         # NEW - credit ledger, balances, costs, functions   (sec.7-8)
      0009_job_queue.sql       # NEW - queue table + lease/complete/fail functions (sec.10)
      0010_billing_events.sql  # NEW - stripe webhook idempotency + rate-limit entitlement cols (sec.9,12)
  backend/
    pyproject.toml             # pinned deps; uv or pip-tools lockfile committed
    Dockerfile                 # single image for api / worker / scheduler
    docker-compose.backend.yml # api, worker, scheduler, redis, clamav; joins supabase network
    app/
      main.py                  # FastAPI factory; middleware order (sec.6.1)
      config.py                # pydantic-settings; every env var typed, fail-fast on boot
      deps.py                  # AuthContext dependency: user, org_id, role, tier, entitlements
      middleware/              # request_id, security_headers, body_limit, rate_limit
      auth/jwt.py              # HS256 verify (JWT_SECRET), aud/exp/email_confirmed checks
      auth/api_keys.py         # thin wrapper over engine.saas issue/verify/revoke
      routes/                  # health, orgs, members, submissions, runs, findings,
                               # credits, billing, webhooks_stripe, api_keys, admin
      services/
        uploads.py             # sniff -> size cap -> clamd -> re-encode -> storage put -> checksum
        credits.py             # cost calc + consume/refund wrappers (SQL functions do the work)
        queue.py               # enqueue_run (same tx as consume_credits)
        stripe_sync.py         # webhook handlers, entitlement resync sweep
        ratelimit.py           # Lua token bucket, fallback limiter
      schemas/                 # pydantic request/response models (reuse engine models)
    worker/
      main.py                  # lease loop, org-fairness dequeue, graceful SIGTERM drain
      executor.py              # wraps engine.saas run path; status transitions; refund-on-system-failure
      providers.py             # circuit breakers + concurrency caps per provider
    scheduler/
      main.py                  # APScheduler: grants, watchdog, retention, reconcile, chain verify, stripe sweep
    tests/                     # unit, integration, security (sec.16)
```

Windows dev note
The engine's working venv is CV-RULE_ENG-NLP/.venv (Windows). The backend runs in Linux containers even in dev, so backend deps are installed in the image, not that venv. The compose file mounts the repo read-only into containers for hot-reload during development; engine code is imported from the mount. First worker boot downloads PP-OCRv6 models to a named volume (~/.paddlex) so OCR warm-up happens once per volume, not per container start.

6 The request lifecycle, end to end

6.1 Middleware order (every request, outermost first)

#	Middleware	Action on failure
1	Request ID + structured access log (request_id threaded into every log line, DB row, and error response)	—
2	Security headers (HSTS, X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy, CSP for docs pages)	—
3	Body size cap (pre-read Content-Length + streaming cap; global 60 MB, tier-tightened at the upload route)	413
4	CORS allowlist (exact origins from env; credentials allowed only for app origin)	CORS reject
5	Authentication: Supabase JWT (HS256 local verify: signature, exp, aud='authenticated', email_confirmed) OR X-API-Key (sha256 lookup, active, scope). Anonymous allowed only for /healthz, /docs, /webhooks/stripe (signature-verified instead)	401
6	Org context resolution: memberships lookup (Redis-cached 60 s), role attached; requested org validated against membership	403
7	Rate limiting (§9): per-IP, per-user, per-org, per-key buckets	429 + Retry-After
8	Route handler: pydantic strict validation → role check → entitlement check → business logic	422 / 403 / 402

6.2 An analysis run, step by step

Step	What happens	Failure surface
1. Submit	POST /v1/submissions (multipart file, or JSON text/html). Auth + org + role (reviewer+) + rate limit pass.	401/403/429
2. Upload pipeline	Sniff real MIME (magic bytes) → extension/MIME allowlist → per-tier size cap → clamd stream scan → images re-encoded via Pillow (strips EXIF/payloads) → sha256 checksum → store at assets/<org>/<submission>/<uuid>.<ext> → assets row (scan_status='clean')	415 bad type, 413 too large, 422 infected (audited)
3. Credit gate + enqueue (ONE transaction)	compute cost = app.credit_cost(kind, cards, tier) → app.consume_credits locks balance row, spends allowance-then-packs, writes ledger → submissions/assets/analysis_runs(status='queued') rows → app.enqueue_run inserts job → COMMIT. Response 202 {run_id, credits_charged, balance}.	402 insufficient_credits (rejected runs recorded + audited, per existing quota contract)
4. Lease	Worker leases via FOR UPDATE SKIP LOCKED honouring tier priority + org fairness + org concurrency cap (§10.2); heartbeats every 30 s.	crash → lease expires → watchdog requeues
5. Execute	executor calls the engine.saas run path: tier fetched from DB via app.org_tier (never from the job payload), statuses running_cv → running_nlp → running_llm → reconciling stream to UI via Realtime; layer_outputs, findings, usage_events persisted; audit events at every edge.	provider outage → backoff/retry, circuit breaker (§10.3)
6. Terminal	complete | human_review | failed. human_review is first-class (migration 0007): UI shows instructions; POST /v1/runs/{id}/resume runs one steered round (new run row linked via resumed_from_run_id, 0 credits, tokens metered).	3 attempts exhausted → DLQ + status failed + automatic credit refund (reason 'refund .system_failure') + audit + alert
7. Read	GET /v1/runs/{id}/report returns the persisted EngineRunResult; findings queryable/approvable; report links to provenance (models, prompt, corpus hash, ruleset versions).	—

Idempotency rule
All side effects key on run_id / event id: a retried job must not double-charge credits (charge happens once at enqueue, not in the worker), must not duplicate findings (worker upserts by run), and a replayed Stripe webhook must no-op (unique event id, sec.12.2). Clients may send Idempotency-Key on POST /v1/submissions; the API stores the key-to-run mapping for 24 h and replays the original 202 response on duplicates.

7 Data model additions (migrations 0008–0010)

Existing schema (0001–0007) already covers tenancy, subscriptions, pipeline, findings, usage_events, hash-chained audit_logs, api_keys, plan_entitlements, storage policies, realtime publication, and engine correlation columns. Three new migrations complete the backend:

0008_credits.sql — the credit system

```
-- Balances: one lockable row per org (SELECT ... FOR UPDATE is the concurrency control)
create table if not exists credit_balances (
  org_id uuid primary key references orgs(id) on delete cascade,
  subscription_credits int  not null default 0 check (subscription_credits >= 0),
  purchased_credits    int  not null default 0 check (purchased_credits    >= 0),
  cycle_start date, cycle_end date,          -- current allowance window (from Stripe period)
  updated_at timestamptz not null default now()
);

-- Append-only ledger: the source of truth an auditor reads
create table if not exists credit_ledger (
  id bigserial primary key,
  org_id uuid not null references orgs(id) on delete cascade,
  delta int not null,                        -- + grants/refunds, - consumption
  bucket text not null check (bucket in ('subscription','purchased')),
  reason text not null check (reason in ('grant.cycle','grant.pack','grant.manual',
        'consume.run','refund.system_failure','adjust.manual','expire.cycle')),
  run_id uuid references analysis_runs(id),
  stripe_ref text,                           -- checkout session / event id for grants
  balance_after int not null,                -- bucket balance after this entry
  actor uuid, created_at timestamptz not null default now()
);
create unique index if not exists uq_ledger_pack on credit_ledger (stripe_ref)
  where reason = 'grant.pack';               -- pack grant idempotency
create unique index if not exists uq_ledger_cycle on credit_ledger (org_id, reason, stripe_ref)
  where reason = 'grant.cycle';              -- one grant per org per cycle (stripe_ref = cycle key)
```

```
-- Published price list (flat cost model, D4)
create table if not exists credit_costs (
  asset_kind text not null check (asset_kind in ('text','image','html','carousel_card')),
  tier text not null,
  credits int not null check (credits > 0),
  primary key (asset_kind, tier)
);

alter table plan_entitlements
  add column if not exists credits_per_month int,          -- null = custom/uncapped (enterprise)
  add column if not exists api_rpm int, add column if not exists api_burst int,
  add column if not exists max_concurrent_runs int not null default 1,
  add column if not exists max_upload_mb int not null default 10,
  add column if not exists credit_packs_allowed boolean not null default false;

-- Functions (security definer, owner role; callable only by service paths)
--  app.credit_cost(org, kind, cards)          -> int     (reads org tier + credit_costs)
--  app.consume_credits(org, run, cost)        -> jsonb   {allowed, charged, sub_bal, pack_bal}
--       locks credit_balances row; spends subscription bucket first, then purchased;
--       inserts ledger rows; raises/returns allowed=false if insufficient. ATOMIC.
--  app.grant_cycle_credits(org, cycle_key)    -> void    idempotent per cycle; expires old
--       allowance (ledger 'expire.cycle'), sets subscription_credits = tier allowance
--  app.grant_purchased_credits(org, n, ref)   -> void    idempotent by stripe_ref
--  app.refund_credits(org, run, n, reason)    -> void    system-failure refunds only
--  app.credit_balance(org)                    -> jsonb   for UI / GET /v1/credits

-- RLS: members read own-org balances + ledger; ALL writes service-role-only (no policies).
-- Ledger + trigger: append-only (block UPDATE/DELETE for every role, like audit_logs).
-- Every function call also writes app.log_audit_event(...) at its commit point.
```

0009_job_queue.sql — durable queue

```
create table if not exists job_queue (
  id bigserial primary key,
  run_id uuid not null unique references analysis_runs(id) on delete cascade,
  org_id uuid not null, tier text not null,
  priority int not null,                     -- enterprise 0 < design 10 < pro 20 < hobby 30 < free 40
  status text not null default 'queued' check (status in ('queued','leased','done','dlq')),
  attempts int not null default 0, max_attempts int not null default 3,
  not_before timestamptz not null default now(),        -- backoff gate: 60s, 300s, 1500s
  locked_by text, lease_expires_at timestamptz,         -- worker id + 300s lease
  last_error text, created_at timestamptz not null default now()
);
create index if not exists ix_jq_dequeue on job_queue (status, not_before, priority, created_at);
-- Functions: app.enqueue_run(run) / app.lease_job(worker_id) (SKIP LOCKED + org concurrency
-- + org fairness, sec.10.2) / app.heartbeat_job(id, worker_id) / app.complete_job(id)
-- / app.fail_job(id, err)  -> requeue with backoff or dlq (returns final disposition so the
-- worker triggers refund + alert on dlq). RLS: service paths only; members never see this table.
```

0010_billing_events.sql — webhook idempotency

```
create table if not exists billing_events (
  id text primary key,                       -- Stripe event id (evt_...)
  type text not null, payload jsonb not null,
  processed_at timestamptz, error text, received_at timestamptz not null default now()
);   -- insert-or-skip on receipt; handler marks processed_at; replay = no-op (sec.12.2)
```

Deprecation: 0008 marks `app.consume_scan_quota()` / `app.scans_used()` as deprecated (kept for rollback one release, comment in SQL); `app.check_quota()` is rewritten to report credit balance so existing callers keep working. `scans_per_month` stays as a column but is no longer enforced.

8 The credit system

8.1 Currency definition

Credits are the single usage currency. A run's price is **published, flat, and known before execution**: cost = base(asset kind) × multiplier(tier model class). Base prices: text 1 · image 2 · html 2 · carousel 2 + 1 per card beyond the first (cards capped at 10). Tier multipliers track model cost: free ×1, hobby ×2, pro ×5, design ×10, enterprise ×10. Examples: a free text scan = 1 credit; a pro image scan = 10; a design 3-card carousel = 40. Operator resumes of human_review runs cost 0 credits (same creative — preserves the existing quota contract) with tokens still metered.

8.2 Allowances, packs, and consumption order

Tier	Monthly allowance (credits)	≈ image scans/mo	Packs purchasable	Concurrent runs
free	25	12	no	1
hobby	400	100	yes	1
pro	6,000	600	yes	3
design	50,000	2,500	yes	6
enterprise	custom (null = contract-governed, still metered)	custom	yes	10 default, custom

- **Allowance bucket** resets to the tier amount at each billing-cycle rollover (grant is idempotent per cycle key = Stripe current_period_start). Unused allowance expires (ledger reason 'expire.cycle') — no rollover.
- **Purchased bucket** (packs: Starter 500 · Growth 2,500 · Scale 12,000 — Stripe one-time prices) never expires, survives tier changes, is consumed only after the allowance bucket hits zero. Paid tiers only.
- **Hard stop at zero:** POST /v1/submissions returns 402 {code:'insufficient_credits', needed, balance, packs_url}. The rejected attempt is recorded and audited (run.quota_rejected pattern already exists). Nothing enqueues, so expensive layers never start unpaid — the credit gate is also the cost-ceiling defence.
- **Tier change mid-cycle:** upgrade grants the new tier's full allowance immediately (fresh cycle key, old allowance expired first); downgrade applies at period end (next grant = new tier). Both audited.

8.3 Atomicity & integrity invariants (non-negotiable)

- consume_credits runs under SELECT ... FOR UPDATE on the org's credit_balances row — two parallel submissions can never overspend; the race test in §16 proves it under 50-way concurrency.
- Charge and enqueue commit in the same transaction: an org is never charged without a queued run, and never queued without a charge.
- The ledger is append-only (trigger blocks UPDATE/DELETE for every role, mirroring audit_logs); balance rows are derived state — the nightly reconcile job re-sums ledger deltas per org and alerts on any drift from credit_balances.
- Refunds are system-initiated only, for runs that fail on OUR side (llm_error after retries, infra failure, DLQ). A completed review the user dislikes, or a human_review verdict, is not refundable — the product delivered its work.
- Every grant/consume/refund/adjust writes both a ledger row and a hash-chained audit_logs event; app.verify_audit_chain() runs nightly.

8.4 What users see

GET /v1/credits returns {subscription_credits, purchased_credits, cycle_start, cycle_end, price_list} and the paginated ledger. The price list is public marketing material — the same credit_costs table renders the pricing page, so what is published is literally what is enforced. usage_events continues to record real token spend per run for margin analytics (cost model feedback loop — playbook §9 note: re-price allowances from real cost data before launch; the numbers above are

launch placeholders in the same spirit as the playbook's).

8.5 The one engine touch-point

`engine/saas.py` → `SupabaseDataPlane.consume_quota()` currently calls `app.consume_scan_quota(org, run)`. It gains a sibling `consume_credits(org, run, cost)` calling the new SQL function, and `run_for_user` accepts the pre-computed cost from the API layer. QuotaExceededError maps to HTTP 402. No other engine code changes; `python -m engine.cli saas-selftest` is extended with credit-path checks (§16).

9 Rate limiting & abuse control

Three independent layers, so no single bypass exposes cost or data. Credits are the cost wall (§8); rate limits are the abuse wall; concurrency caps are the fairness wall.

9.1 Layer 1 — edge (reverse proxy)

- Connection caps, request/header timeouts (slowloris), 60 MB absolute body cap, TLS termination, HSTS. Optional WAF (Coraza rules or Cloudflare proxied DNS) in front for launch.

9.2 Layer 2 — API middleware (Redis token buckets, atomic Lua)

Bucket key	Applies to	Default limits (per plan_entitlements where tiered)
ip:{addr}	Unauthenticated routes (auth callbacks, docs, 404s)	30 req/min; signup/login attempts 10/min/IP
user:{sub}	Browser JWT traffic	free 60 · hobby 120 · pro 120 · design 240 · ent 600 req/min
org:{id}:api	Programmatic /v1 via API keys (Pro+ entitlement api_access)	pro 60 rpm burst 120 · design 300 rpm burst 600 · ent custom (default 1,000)
key:{prefix}	Individual API key (optional per-key cap ≤ org cap, set at issuance)	defaults to org limit
org:{id}:submit	Submission creations specifically (defence for the expensive route)	free 5/day · hobby 50/day · pro 500/day · design/ent unlimited (credits govern)

- Responses carry X-RateLimit-Limit / -Remaining / -Reset; 429 includes Retry-After. Limits resolve from plan_entitlements (new columns api_rpm/api_burst), cached in Redis 60 s, invalidated by the Stripe webhook on tier change.
- Repeated 429 abuse (>10/min sustained) sets a temporary Redis ban (15 min TTL) and writes an audit event; permanent offenders are handled by key revocation (existing api_keys.revoke).

9.3 Layer 3 — work-level throttles

- **Per-org concurrent runs** (§8.2 table) enforced at dequeue — a job is leased only if the org's active-run count is below its cap; otherwise the worker skips to the next org's job (fairness, §10.2).
- **Per-provider global concurrency** protects Anthropic/OpenRouter rate limits and the spend ceiling: worker-pool semaphores (Redis) cap parallel in-flight LLM calls per provider (env: PROVIDER_MAX_PARALLEL_ANTHROPIC / _OPENROUTER). Provider 429/5xx → exponential backoff inside the run, circuit breaker across runs (§10.3) — users see 'queued', never a provider error.
- **Free-tier abuse:** email verification already required before runs (engine gate); add disposable-email domain blocklist at signup, per-IP signup throttle (5/day), Cloudflare Turnstile on signup, and a weekly report of multi-org-per-IP/device patterns. Free tier has no API access (entitlement) and 1 concurrent run.

9.4 Failure mode

If Redis is unreachable: each api replica falls back to an in-process token bucket at 50% of configured limits (conservative but available) and pages ops; the credit gate (Postgres) still hard-stops all spend, so the cost wall never depends on Redis. Rate limiting fails degraded-closed for /v1 API keys (strictest audience) and degraded-open for browser sessions (UX-preserving) — both logged with request_id.

10 Job queue & worker design

10.1 Why Postgres, not Redis/BullMQ/Celery-broker

- BullMQ is Node — the worker is Python (D1). A Redis broker would also split the enqueue across two systems, losing the single-transaction guarantee of D3: with Postgres, credit spend + run row + job row commit atomically, so a crash between 'charged' and 'queued' is impossible by construction.
- Throughput ceiling of SKIP LOCKED queues (hundreds of jobs/sec) exceeds any plausible load by orders of magnitude — each job then runs seconds-to-minutes of engine work. The JobQueue interface in services/queue.py is narrow (enqueue/lease/heartbeat/complete/fail) so a Celery/SQS swap later is contained.

10.2 Dequeue discipline

```
app.lease_job(worker_id):
  with one statement, atomically:
    pick the oldest queued job whose not_before <= now()
      AND whose org has active_runs < max_concurrent_runs(tier)     -- concurrency wall
    ordered by priority asc (tier), then least-recently-served org  -- fairness: round-robin
      across orgs, so one org's burst cannot starve others           -- within a priority band
    FOR UPDATE SKIP LOCKED LIMIT 1
  set status='leased', locked_by=worker_id, lease_expires_at=now()+300s, attempts+=1
```

- Heartbeat every 30 s extends the lease; a worker that dies mid-run stops heartbeating and the scheduler's watchdog returns the job to 'queued' after lease expiry (crash recovery). The engine's persisted layer_outputs let a retried run skip re-OCR (existing capability).
- Enterprise option later: a dedicated worker pool filtered to enterprise jobs — the 'dedicated throughput' differentiator the playbook prescribes; requires only a lease filter, no schema change.

10.3 Failure taxonomy

Failure	Handling	Credits
Transient (provider 429/5xx/timeout, network)	In-run tenacity retries; then job retry with backoff 60 s → 300 s → 1,500 s (max_attempts 3). Circuit breaker per provider: open after 5 consecutive failures, half-open probe every 60 s; while open, jobs for that provider stay queued (status visible to user as 'queued').	kept (run will complete)
Permanent system failure (attempts exhausted, engine crash, malformed provider output after engine's own corrective retry)	job → dlq; run → failed; automatic app.refund_credits(reason 'refund.system_failure'); audit event; ops alert with last_error and request/run ids.	refunded
llm_error surfaced by engine (missing key, provider auth)	treated as system failure path above — user never pays for our misconfiguration.	refunded
human_review verdict	NOT a failure: first-class terminal status; operator resume path (0 credits) per existing contract.	kept
Infected upload / validation reject	never reaches the queue (rejected pre-charge in the upload pipeline).	never charged
Stuck run (heartbeat silent, lease expired)	watchdog requeues (attempts++), same taxonomy applies from there.	per outcome

10.4 Worker process model

- One leased run per worker process at a time (engine layers are CPU-heavy in L1 and IO-heavy in L3); scale = container replicas. PaddleOCR models load once per process and stay warm. SIGTERM → stop leasing, finish or cleanly release the current job, exit (compose stop_grace_period 120 s) — deploys never kill runs.

11 API surface

All endpoints under /v1, JSON, OpenAPI-documented. Two auth schemes: browser JWT (all tiers) and X-API-Key (Pro+ entitlement, scopes: submit / read / admin). Role floor noted per group; entitlements checked server-side on every gated action.

Endpoint	Method(s)	Auth / role floor	Purpose
/healthz · /readyz · /metrics	GET	none · none · internal	liveness; readiness (DB+Redis ping); Prometheus (proxy-blocked publicly)
/v1/orgs · /v1/orgs/me	POST / GET	JWT · member	create via app.create_org() (atomic org+owner+free sub); current org+tier+role
/v1/orgs/members	GET, POST, PATCH, DELETE	admin+ (seat cap enforced from entitlements)	invite, role change, remove; all audited
/v1/credits · /v1/credits/ledger	GET	member	balance, cycle, price list; paginated ledger
/v1/credits/checkout	POST	admin+ · packs entitlement	Stripe Checkout session for a credit pack
/v1/submissions	POST	reviewer+ · credits	multipart upload or JSON text/html; returns 202 {submission_id, run_id, credits_charged, balance}; Idempotency-Key honoured
/v1/submissions · /{id}	GET	member	list (cursor-paginated, org-scoped) / detail with assets
/v1/runs/{id} · /v1/runs	GET	member	status + timings + provenance; list with filters (status, date)
/v1/runs/{id}/report	GET	member	full persisted EngineRunResult (verdict, findings, rounds, provenance)
/v1/runs/{id}/resume	POST	reviewer+	operator guidance for human_review runs; new linked run, 0 credits
/v1/findings · /{id}	GET / PATCH	member / reviewer+	list per run/submission; approve | flag | dismiss (audited)
/v1/api-keys · /{id}	GET/POST · DELETE	admin+ · api_access entitlement	issue (secret shown once, sha256 at rest — existing impl), list prefixes, revoke
/v1/billing/checkout · /portal	POST	owner/admin	subscription checkout; Stripe Billing Portal link
/webhooks/stripe	POST	Stripe signature	the ONLY billing writer (§12)
/v1/admin/*	various	internal staff allowlist + audit	DLQ inspect/requeue, org lookup, manual credit adjust (dual-logged)

Error model

```
{ "error": { "code": "insufficient_credits", "message": "Run costs 10 credits; balance is 4.",
             "request_id": "req_7f3a...", "details": {"needed": 10, "balance": 4} } }
400 validation_error | 401 unauthenticated | 402 insufficient_credits | 403 forbidden(role/entitlement)
404 not_found (also returned for cross-org ids - never reveal existence) | 409 conflict/concurrency
413 payload_too_large | 415 unsupported_media_type | 422 unprocessable(infected_file, bad_asset)
429 rate_limited (+Retry-After) | 5xx internal (no stack traces, request_id only)
```

Versioning: breaking changes require /v2; additive fields are non-breaking. The OpenAPI JSON is served at /v1/openapi.json and the generated TS client is committed to frontend/ tooling.

12 Billing: Stripe integration

12.1 Catalogue

- Subscriptions: hobby, pro, design as self-serve Products with monthly+annual Prices (STRIPE_PRICE_* env mapping to tiers); enterprise via sales-led checkout links. Free tier = no Stripe object (default subscription row from app.create_org).
- Credit packs: three one-time Prices (Starter 500 / Growth 2,500 / Scale 12,000) with metadata {kind:'credit_pack', credits:N}; sold through Checkout, gated by credit_packs_allowed entitlement.

12.2 Webhook contract (the source of truth)

```
POST /webhooks/stripe            raw body preserved; Stripe-Signature verified (STRIPE_WEBHOOK_SECRET)
 1. insert into billing_events (id, type, payload) on conflict do nothing
    -> if row existed and processed_at is set: return 200 (replay no-op)
 2. handle by type:
    customer.subscription.created|updated|deleted
        -> upsert subscriptions (org via customer metadata: tier, status, period start/end)
        -> if tier changed or new period: app.grant_cycle_credits(org, cycle_key)
        -> invalidate Redis entitlement cache for org
    checkout.session.completed (metadata.kind='credit_pack')
        -> app.grant_purchased_credits(org, credits, session_id)   # idempotent by ref
    invoice.payment_failed / past_due
        -> subscriptions.status sync; grace policy: 7 days read-only warnings, then
           submissions blocked (403 billing_hold) until resolved; data never deleted for this
 3. mark processed_at; any handler error leaves processed_at null -> Stripe retries; alert after 3
```

- Entitlements always resolve from the DB (app.org_tier — the engine already fetches tier per run this way; a subscription row change takes effect on the very next run with no code moves).
- Daily reconcile sweep (scheduler) lists active Stripe subscriptions and diffs against the subscriptions table — webhook delivery is at-least-once but not guaranteed forever; the sweep is the safety net, alerting on drift rather than silently writing.
- Proration on upgrades handled by Stripe defaults; the webhook's tier change grants the new allowance immediately (§8.2). Cancellations: cancel_at_period_end → tier flips to free at the final rollover, purchased credits persist and remain spendable at free-tier pricing.

13 Security architecture (cyber safety)

13.1 Identity & access

- JWTs verified locally (HS256 + rotated JWT_SECRET): signature, exp, aud, email_confirmed. No token → 401 before any handler code. Role lattice owner > admin > reviewer > viewer enforced per route (§11) AND re-checked in DB paths (engine/saas.py already refuses below-reviewer runs).
- API keys: sha256 at rest, prefix-displayed, scoped, revocable, entitlement-gated (existing implementation) — plus per-key rate caps (§9.2) and last_used_at stamping for key-hygiene reviews.
- The privileged DB role (SUPABASE_DB_URL) bypasses RLS: it exists only in api/worker/scheduler env, every statement sets org_id explicitly, and the red-line RLS test plus code review guard the discipline. The anon key is the ONLY key the frontend ever holds.

13.2 Transport & perimeter

- TLS 1.2+ everywhere, auto-renewed (Caddy); HSTS; secure/httpOnly/SameSite cookies (Supabase auth); strict CORS allowlist; security headers middleware (§6.1). Only 80/443 (proxy) and key-only SSH reachable on the VPS: ufw default-deny, Postgres/Studio/Redis/clamd internal-only. Docker networks: proxy→api and api/worker→data-plane segments; no container talks to the internet except worker→providers and api→Stripe through the egress allowlist.

13.3 Untrusted input (the dangerous boundary)

- **Uploads:** extension AND sniffed-MIME allowlist (png, jpeg, webp, gif, pdf, txt, html, mp4) → per-tier size caps (10/20/30/50/50 MB) → clamd stream scan (EICAR test in CI) → raster images re-encoded with Pillow (strips EXIF, embedded payloads, polyglot tricks) → random storage keys under assets/<org>/<submission>/ → served ONLY via 60 s signed URLs from the storage origin, never proxied through the app, never rendered server-side.
- **HTML assets:** parsed (BeautifulSoup, engine's existing path), never executed or headless-rendered in v1. When landing-page URL ingestion ships (Phase B): DNS-resolve then block private/link-local/metadata CIDRs (127/8, 10/8, 172.16/12, 192.168/16, 169.254/16, ::1, fc00::/7), re-validate every redirect hop (max 3), fetch via EGRESS_PROXY_URL allowlist, 10 s timeout + 5 MB cap, and render only inside a network-isolated Playwright container whose only route is the egress proxy — the playbook's SSRF box, implemented verbatim.
- **Prompt injection:** the engine's existing containment stands — ad text enters Layer 3 as inert data inside a JSON block, system instructions stay in the system turn, feedback is engine/operator-authored, unknown citation ids are dropped loudly, and Layer 4's ~50 deterministic checks re-verify every review. The backend adds: model output is never executed, findings text is stored raw and escaped at render, and no LLM output triggers side effects without rule-engine verification.

13.4 Secrets & supply chain

- Secrets live in per-environment .env files outside the repo (and a secrets manager on the VPS); .env.example lists keys only. Provider keys and SUPABASE_DB_URL exist solely in backend containers. gitleaks + pip-audit + pinned lockfile + Dependabot in CI; images pinned by digest; harden.py rotation runbook for JWT/DB secrets with documented client-update order (supabase.md §3).

13.5 Audit

- Hash-chained, append-only audit_logs (existing, migration 0003) receive every security-relevant edge: auth failures (sampled), membership/role changes, key issue/revoke, submission/run lifecycle, finding approvals, billing/tier changes, every credit ledger mutation, admin actions (dual-logged with staff identity), data-deletion executions. Nightly app.verify_audit_chain() — a broken chain pages immediately. No secrets or raw PII in meta.

14 Data privacy & compliance

14.1 Data inventory & classification

Class	Examples	Store	Retention default
Account PII	auth.users email, membership records	Postgres (RLS)	life of account + 30 d
Customer content	uploaded creatives (may embed PII in ad imagery/copy)	Storage bucket assets/ (private)	free 30 d · paid 90 d · configurable for enterprise; then scheduled deletion
Derived work product	layer_outputs, findings, reports, provenance	Postgres (RLS)	life of account (the auditable asset customers pay for)
Financial	Stripe customer/sub ids ONLY (no card data ever touches our stack — SAQ-A posture)	Postgres	life of account
Telemetry	logs/metrics keyed by request_id/run_id, no content bodies	log store	30 d
Audit	hash-chained events	Postgres, append-only	≥ 2 years

14.2 Controls

- **Minimisation to providers:** only OCR-extracted text, measurements, and regulation excerpts reach LLM APIs — never the raw image file. A pre-egress redaction pass masks obvious direct identifiers (SSN/card-number/email patterns) from extracted text as a belt-and-braces guard. Anthropic/OpenRouter API-tier no-training defaults verified and pinned in the DPA/subprocessor list; model IDs pinned in plan_entitlements.
- **Erasure (GDPR/CCPA-ready even though launch is US-only):** one documented runbook + admin endpoint: delete org → FK cascade clears DB rows → storage prefix assets/<org>/ purged → Stripe customer deleted → provider-side deletion requests filed → completion recorded in audit (the audit event persists; content does not). Tested in §16; target ≤ 30 days, actual: same-day.
- **Encryption:** TLS in transit everywhere (including container-to-Postgres in prod); at rest via LUKS-encrypted VPS volumes for DB + storage; backups encrypted with age to offsite object storage (R2/S3), 7 daily + 4 weekly retained; nightly pg_dump now, WAL/PITR (pgbackrest) before GA — restore tested before launch, RPO ≤ 24 h documented (supabase.md §7).
- **Tenant isolation:** RLS everywhere + explicit org scoping in privileged paths + cross-org ids answered with 404 (existence never revealed) + storage policies already org-prefixed (migration 0005). The RAG corpus stays locked from the public API (migration 0006).
- **Data residency:** all customer data on the self-hosted stack (US region VPS at launch) — the self-hosting rationale; document in the DPA alongside subprocessors (Anthropic, OpenRouter, Stripe, host, R2).
- **Product posture:** reports remain decision support, not legal advice — the disclaimer renders in every report and API response footer (existing engine framing; liability posture per playbook §16).

15 Observability, reliability & ops

15.1 Telemetry

- structlog JSON to stdout: request_id → run_id correlation from API through worker through each layer (engine already returns timings_ms per phase). Sentry on api+worker+scheduler with release tags. Prometheus /metrics: queue depth & oldest-job age, runs by status/tier, layer durations, provider latency/error/429 counts, circuit-breaker state, rate-limit hits, credit spend/refunds, webhook lag, DLQ size.

15.2 Alerts that page

- DLQ > 0 · oldest queued job > 10 min · audit chain broken · ledger reconcile drift · webhook failures · provider breaker open > 5 min · readyz failing · disk > 80% · backup job missed · Redis down (fallback limiter active).

15.3 Cost control

- usage_events per run per model (exists) rolled into an internal margin dashboard: credits charged vs token cost per tier — the feedback loop that re-prices §8.2 before launch. Per-provider concurrency caps + credit hard-stop + free-tier throttles are the runaway-spend defences; a monthly provider budget alarm is the last resort.

15.4 Deployment path (D6)

```
NOW (this machine):  docker compose -f docker-compose.backend.yml up -d
  - joins the existing supabase network (external network ref)
  - .env.backend written AFTER supabase/harden.py rotation
  - frontend dev: next dev -> http://localhost:8000 (Kong) + http://localhost:8080 (api)

LAUNCH (Linux VPS, e.g. Hetzner CPX41+):
  1. Provision: ufw default-deny (allow 80/443 + key-only SSH), fail2ban, unattended-upgrades,
     LUKS data volume, Docker
  2. Move stack: rsync supabase-project volumes (or pg_dump/restore) -> verify RLS test +
     verify_audit_chain + saas-selftest on the VPS BEFORE cutover
  3. Same two compose files up; Caddy overlay (COMPOSE_FILE=...caddy.yml) for TLS;
     DNS app./api./supabase. -> VPS; SMTP (Resend) creds in auth env (supabase.md sec.5)
  4. Backups cron -> encrypted offsite; uptime monitor on /readyz; pen-test checklist (Appendix C)
  Rollback: previous images kept one release back (docker tags); DB migrations forward-only
  with pre-deploy dump; deploys drain workers via SIGTERM grace (sec.10.4) - zero dropped runs.
```

15.5 SLOs (initial)

- API availability 99.5% · submission-accepted p95 < 800 ms · text-run completion p95 < 3 min, image p95 < 6 min (tier LLM dependent) · webhook-to-entitlement < 60 s · support: community/email per tier, Enterprise SLA formalised at contract.

16 Testing & verification

Suite	Proves	Gate
Existing engine suites (184 + 67 + selftests)	layers, wiring, saas data plane — must stay green untouched	CI, every PR
RLS isolation test (supabase/tests/)	Org A cannot read/list/mutate Org B — through API and DB	RED-LINE: blocks merge
Credit race test (NEW)	50 parallel submissions against a balance funding 10: exactly 10 succeed, balance never negative, ledger sums to balance	RED-LINE: blocks merge
Credit lifecycle unit tests	cost matrix, bucket order (allowance→packs), cycle grant idempotency (same cycle_key twice = one grant), expire-on-rollover, refund-only-on-system-failure, pack grant idempotency by stripe_ref	CI
Auth/authz matrix	every route × {anon, viewer, reviewer, admin, owner, wrong-org, revoked-key, unverified-email} → expected status; cross-org ids return 404	CI
Rate-limit integration	429 + headers at configured rpm; per-key cap ≤ org cap; Redis-down fallback behaves per §9.4	CI (compose)
Upload security	EICAR rejected + audited; MIME spoof (exe renamed .png) rejected; oversize 413; polyglot re-encode; signed-URL expiry	CI (compose)
Queue chaos	kill worker mid-run → lease expires → requeued → completes; 3 forced failures → DLQ + refund + alert; SIGTERM drain loses nothing	CI (compose)
Stripe (test clock + CLI)	checkout upgrades tier & grants allowance within seconds; replayed webhook no-ops; pack purchase credits exactly once; past_due grace; cancel → free at period end	staging
E2E (stub LLM)	signup → org → submit text+image → realtime statuses → report → approve finding → resume human_review run — extends python -m engine.cli saas-selftest with credit assertions	CI nightly + staging
Load	100-submission burst across 10 orgs: fairness holds, no double-charge, queue drains, breaker never trips on stub	staging
Security suite	SSRF probes (when URL ingestion ships), injection goldens through engine, gitleaks, pip-audit; external pen test pre-GA	staging / pre-launch

Definition of done for the whole build

All CI suites green including both red-line tests; staging E2E and Stripe suites pass; restore-from-backup rehearsed; Appendix C checklist fully ticked; harden.py verified (grep for default secrets returns nothing). Only then is public traffic pointed at the VPS.

17 Build order for Claude Code (phases & gates)

Sequential; each gate is objective and scriptable. Do not reorder: the credit system precedes any route that could spend money, and security middleware precedes any route at all.

Phase	Build	Verification gate (must pass to proceed)
P0. Prerequisite	Run supabase/harden.py; write backend/.env from rotated secrets	grep 'your-super-secret' stack .env → empty; old anon/service keys return 401; saas-selftest still passes
P1. Scaffold	backend/ package, Dockerfile, compose (api+redis+clamav), config.py fail-fast settings, /healthz /readyz, request-id + security-header middleware, CI skeleton (lint, mypy, pytest, gitleaks, RLS test wired)	compose up → /readyz 200 (DB+Redis ping); CI green on empty suite; existing engine tests still green
P2. Migrations	0008 credits + 0009 job_queue + 0010 billing_events via apply.sh; seed credit_costs, allowances, rate-limit entitlement columns	apply.sh idempotent (run twice); credit race test passes; ledger append-only trigger blocks UPDATE/DELETE as superuser-adjacent role
P3. AuthN/Z	jwt.py, api_keys wrapper, deps.py AuthContext, org resolution + Redis cache	auth/authz matrix suite green; cross-org 404s verified
P4. Rate limiting	Lua buckets, entitlement-driven limits, headers, fallback limiter, ban list	rate-limit integration suite green incl. Redis-down mode
P5. Upload + submit	uploads.py pipeline, submissions route, consume_credits + enqueue single-transaction path, Idempotency-Key store	upload security suite green; EICAR + spoof rejected; 402 path audited; charge+enqueue atomicity proven by fault injection between statements (must roll back together)
P6. Worker	lease loop, executor over engine.saas (consume_credits swap §8.5), heartbeat, watchdog (scheduler stub), breaker, refund-on-DLQ	queue chaos suite green; extended saas-selftest green E2E with stub LLM; realtime statuses observed
P7. Read APIs	runs, reports, findings (+PATCH), credits/ledger, orgs/members, resume route	E2E suite green; OpenAPI reviewed; TS client generates
P8. Stripe	checkout/portal routes, webhook handler + billing_events, grants wiring, entitlement cache invalidation, grace policy	Stripe test-clock suite green; replay no-op proven; tier change reflected in next run's model (query provenance)
P9. Scheduler	APScheduler jobs: cycle grants sweep, watchdog, retention deletion, ledger reconcile, chain verify, Stripe drift sweep	simulated period rollover grants exactly once; retention deletes only past-window rows (dry-run mode first); reconcile detects an injected drift
P10. Programmatic API	API-key auth on /v1, scopes, per-key caps, docs page	key-scoped rate caps enforced; revoked key 401s within cache TTL
P11. Observability	structlog, Sentry, /metrics, alert rules, margin dashboard queries	alert dry-run fires on injected DLQ item; dashboards render
P12. Hardening + launch	load suite, security suite, backup+restore rehearsal, VPS migration (§15.4), Appendix C tick-through, pen test	Definition of done (§16) fully satisfied

Estimated shape: P1–P6 is the critical path (the spine); P7–P10 parallelise if needed. Every phase lands as a reviewed PR with its gate output pasted in the description.

A Appendix A — Environment variables

```
# ---- backend/.env.example (values live in per-environment secret stores; never committed) ----
# Data plane
SUPABASE_URL=                       # Kong URL
SUPABASE_ANON_KEY=                  # frontend only (listed for completeness)
SUPABASE_SERVICE_ROLE_KEY=          # SERVER ONLY - storage/admin API calls
SUPABASE_DB_URL=                    # privileged Postgres role for engine.saas (bypasses RLS)
DATABASE_URL_API=                   # pooler :6543 transaction mode (API request pool)
DATABASE_URL_WORKER=                # pooler :5432 session mode (worker/scheduler)
DATABASE_URL_MIGRATIONS=            # socat :54322 direct (apply.sh / DDL only)
SUPABASE_JWT_SECRET=                # HS256 verify (rotated by harden.py)

# App
API_PORT=8080  CORS_ALLOWED_ORIGINS=https://app.adcoherentlabs.com
REDIS_URL=redis://:pass@redis:6379/0
CLAMAV_HOST=clamav  CLAMAV_PORT=3310
RATE_LIMIT_FAIL_MODE=degraded       # sec.9.4
MAX_BODY_MB=60                      # edge cap; per-tier caps come from plan_entitlements

# Workers
WORKER_ID_PREFIX=wk  LEASE_SECONDS=300  HEARTBEAT_SECONDS=30
PROVIDER_MAX_PARALLEL_ANTHROPIC=4  PROVIDER_MAX_PARALLEL_OPENROUTER=8
STOP_GRACE_SECONDS=120

# Providers (server only; pinned models resolve from plan_entitlements / engine tiers)
ANTHROPIC_API_KEY=                  # pro / design / enterprise
OPENROUTER_API_KEY=                 # free / hobby (Qwen via OpenRouter)
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
EGRESS_PROXY_URL=                   # SSRF allowlist proxy (URL-ingestion phase)

# Billing
STRIPE_SECRET_KEY=  STRIPE_WEBHOOK_SECRET=
STRIPE_PRICE_HOBBY=  STRIPE_PRICE_PRO=  STRIPE_PRICE_DESIGN=  STRIPE_PRICE_ENTERPRISE=
STRIPE_PRICE_PACK_STARTER=  STRIPE_PRICE_PACK_GROWTH=  STRIPE_PRICE_PACK_SCALE=

# Ops
SENTRY_DSN=  LOG_LEVEL=info  METRICS_BASIC_AUTH=
BACKUP_TARGET=                      # r2/s3 bucket for encrypted dumps
AGE_RECIPIENT=                      # backup encryption key
```

B Appendix B — Seed tables

credit_costs seed (credits per run)

Asset kind	free (×1)	hobby (×2)	pro (×5)	design (×10)	enterprise (×10)
text	1	2	5	10	10
image	2	4	10	20	20
html	2	4	10	20	20
carousel (2 cards)	3	6	15	30	30
+ each extra card (max 10)	+1	+2	+5	+10	+10
operator resume	0	0	0	0	0

plan_entitlements additions

Tier	credits/mo	api_rpm / burst	concurrent runs	upload MB	submissions/day	packs
free	25	— (no API)	1	10	5	no
hobby	400	— (no API)	1	20	50	yes
pro	6,000	60 / 120	3	30	500	yes
design	50,000	300 / 600	6	50	unlimited*	yes
enterprise	custom (null)	custom (1,000 default)	10 default	50	unlimited*	yes

** credits govern. All numbers are launch placeholders — the margin dashboard (§15.3) re-prices them from observed token cost before GA, per the playbook's margin note. Unchanged from the existing seed: model routing, max output tokens (1.5k/3k/8k/16k/32k), seats (1/2/8/20/custom), frameworks scope, SSO/SLA flags.*

Credit packs

Pack	Credits	Stripe object	Notes
Starter	500	one-time Price, metadata credits=500	paid tiers only; never expires
Growth	2,500	one-time Price, metadata credits=2500	consumed after allowance bucket
Scale	12,000	one-time Price, metadata credits=12000	price points set from cost model

C Appendix C — Non-goals & hard guardrails

- Do NOT modify engine layer code (CV-RULE_ENG-NLP/, rag-engine/, engine/ orchestration). The only sanctioned engine change is the consume_credits swap in engine/saas.py (§8.5).
- Do NOT run the engine inside the API process, ever — analysis is queue-only. The API's job ends at COMMIT of charge+enqueue.
- Do NOT bypass the credit gate or reorder it after enqueue; do NOT charge in the worker.
- Do NOT trust client-supplied org_id, tier, or model — DB is authority (app.org_tier, memberships).
- Do NOT put SUPABASE_SERVICE_ROLE_KEY, SUPABASE_DB_URL, or provider keys anywhere client-reachable, in logs, or in error responses.

- Do NOT render, execute, or headless-browse user-uploaded HTML in v1; URL ingestion waits for the sandboxed fetcher (§13.3).
- Do NOT write migrations that mutate applied files — forward-only, next number, idempotent, backup before destructive DDL.
- Do NOT expose Postgres, Redis, clamd, Studio, or /metrics publicly; only the reverse proxy listens on the internet.
- Do NOT let a Stripe webhook handler write entitlements without signature verification + billing_events idempotency.
- Do NOT delete or rewrite audit_logs or credit_ledger rows — append-only is enforced by trigger and by policy; corrections are new compensating entries.
- Do NOT claim compliance accuracy numbers in product copy until the golden-set evaluation harness (playbook Phase 5) reports them — reports remain decision support, not legal advice.

End of specification. Companion references: ENGINE.md (engine contracts & saas data plane), supabase.md (stack ops, hardening, migrations 0001–0007), AdCoherent_Labs_Build_Playbook.pdf (product phases; this document implements Phases 4, 6, 8, 9 and parts of 10–11 for the backend).